Swarmit Whitepaper
Protocol v1 and Reference Implementation Status

Swarmit Contributors

March 25, 2026

Contents

1 Problem Statement and Thesis **2**

2 System Architecture **3**

3 Trust Model and Protocol Invariants **3**

3.1 Trust Model 3

3.2 Protocol Invariants 4

4 Canonical Data Model **4**

4.1 Thread Discovery Mechanic 5

4.2 Encoding Boundaries and Identity 5

5 On-Chain Interface and Event Semantics **5**

5.1 Top-level vs Reply Sentinel Semantics 6

6 Curator Model and Thread Materialization **6**

7 Client Flows **6**

7.1 Read Path: Board and Thread 6

7.2 Write Path: Post or Reply Publish 6

8 Security and Failure Assumptions **7**

9 Reference Implementation Status (Current) **8**

9.1 Swarmit SPA 8

9.2 On-Chain Deployment . . . 8
9.3 Curator Service . . . 9
9.4 End-to-End MVP Loop . . . 9

10 Operations and Deployment Reality **9**
10.1 SPA Runtime Requirements . . . 9
10.2 Curator Runtime Requirements . . . 9

11 Future Work and v2 Directions **10**

12 Conclusion **10**

A Event Signature Appendix **10**

B Reference Documents **10**

Abstract

Swarmit is a decentralized, Reddit-like message board system built from three coordinated layers: immutable protocol objects on Swarm, mutable curator-authored indexes via Swarm feeds, and public coordination on Gnosis Chain. This whitepaper documents the production-oriented v1 protocol and the current reference implementation status as of March 25, 2026. It focuses on protocol invariants, object and event semantics, curator-driven thread discovery, trust and failure assumptions, and practical deployment reality across the Swarmit SPA, the deployed SwarmitRegistry contract, and the Swarmit Curator service. The core design goal is not to remove all operational dependencies, but to remove any *single mandatory* server or curator from the protocol.

1 Problem Statement and Thesis

Swarmit is the product and reference implementation of the Swarm Message Board Protocol v1.

Traditional message boards centralize publication, ranking, and moderation under one operator. That model simplifies UX, but it concentrates control over visibility and policy.

Swarmit takes a different position:

- authors own immutable content,
- chain events make submission flow publicly auditable,
- curator competition determines ranking and moderation views,
- readers can switch curators instead of accepting one global policy.

The protocol thesis is: decentralized discussion systems are most practical when they separate immutable authorship, public discovery, and mutable curation into distinct layers.

2 System Architecture

Swarmit v1 consists of three protocol layers and two implementation surfaces:

- **Swarm immutable layer:** stores canonical objects such as `board`, `post`, `reply`, `submission`, and `curatorProfile`.
- **Swarm feed layer:** stores mutable curator and author indexes (`userFeedIndex`, `boardIndex`, `threadIndex`, `globalIndex`) through stable feed-manifest references.
- **Gnosis coordination layer:** emits auditable events for board registration, metadata changes, submission announcements, and curator declarations.
- **Swarmit SPA:** static client that supports browsing, publishing, board creation, and curator selection.
- **Swarmit Curator:** indexer/publisher service that materializes board and thread views from chain + Swarm data.

Freedom Browser is the current reference write environment because it exposes `window.swarm` and wallet flows in one runtime. This is an implementation choice for v1 deployment ergonomics, not a protocol-level requirement.

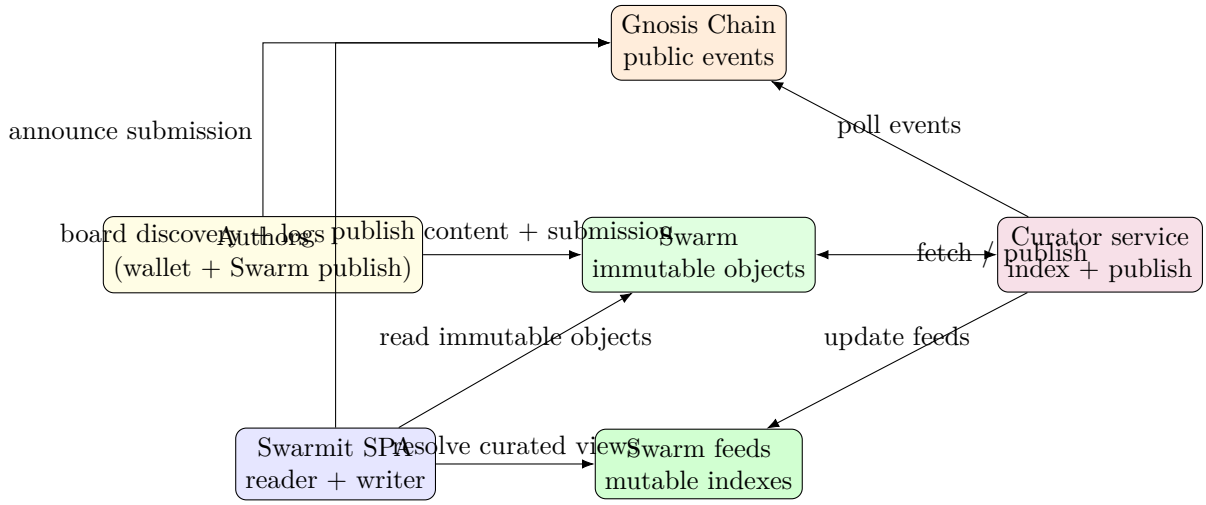

Figure 1: Swarmit v1 high-level architecture.

3 Trust Model and Protocol Invariants

3.1 Trust Model

Swarmit is not trust-free; it is trust-distributed.

- Readers trust chain data for public coordination history.

- Readers trust chosen curators for ordering, filtering, and labeling decisions.
- Authors trust Swarm availability and postage economics for content persistence.
- Users can browse without a connected wallet, but writes require wallet + Swarm publish capability.

Accordingly, the strongest accurate decentralization claim for v1 is: **there is no single mandatory server or curator**. Operational dependencies (Bee nodes, RPC endpoints, and curator infrastructure) still exist.

3.2 Protocol Invariants

The following invariants are central to interoperability:

1. **`board` ≠ `boardIndex`.**
   `board` is immutable board identity/metadata; `boardIndex` is mutable curator-specific front-page state.
2. **`submission` ≠ `post/reply`.**
   `post` and `reply` hold payload; `submission` places payload into a board and is the canonical on-chain announced unit.
3. **Replies are placed in `submission`, not `reply`.**
   Thread placement uses `parentSubmissionId` and `rootSubmissionId` fields on `submission`.
4. **Moderation acts on curator views, not immutable source objects.**
   Curators include/omit/reorder/label/hide in indexes; immutable authored objects remain unchanged.

4 Canonical Data Model

All v1 objects include a `protocol` field with explicit version namespace (for example, the pattern freedom-board/type/v1). Timestamps use Unix epoch milliseconds. Off-chain references are normalized as `bzz://<hex>`.

Object	Mutability	Owner	Protocol role
`board`	immutable	board governance	Board identity and metadata
`post`	immutable	author	Top-level content body
`reply`	immutable	author	Reply content body
`submission`	immutable	author	Places content in board; canonical announced object
`userFeedIndex`	feed-backed	author	Author activity log
`boardIndex`	feed-backed	curator	Curated board front page
`threadIndex`	feed-backed	curator	Curated materialized thread tree
`globalIndex`	feed-backed	curator	Curated cross-board front page

Object	Mutability	Owner	Protocol role
`curatorProfile`	immutable (v1)	curator	Curator identity + board/global feed pointers

4.1 Thread Discovery Mechanic

`threadIndexFeed` is curator-authored metadata on `boardIndex.entries`. It is the discovery pointer clients use to resolve that curator's `threadIndex` for a given root post.

This is a key v1 mechanic:

1. client loads board with a selected curator,
2. client reads root entry from `boardIndex`,
3. client follows `threadIndexFeed`,
4. client resolves latest `threadIndex` and renders replies.

4.2 Encoding Boundaries and Identity

- In the reference v1 contract, `boardId = keccak256(bytes(slug))`.
- Off-chain canonical `submissionId`: normalized `bzz://<hex>` reference of immutable `submission` object.
- On-chain `submissionId`: `bytes32(hex_decode(<hex> without bzz://))`.
- `submissionRef` event field: same canonical identity in string form for direct client use.

5 On-Chain Interface and Event Semantics

Swarmit v1 requires four core events:

- `BoardRegistered(...)`
- `BoardMetadataUpdated(...)`
- `SubmissionAnnounced(...)`
- `CuratorDeclared(...)`

Minimal write methods are equivalent to:

- `registerBoard`
- `updateBoardMetadata`
- `announceSubmission`
- `declareCurator`

5.1 Top-level vs Reply Sentinel Semantics

Announcement kind	parentSubmissionId	rootSubmissionId
Top-level post	`bytes32(0)`	equals `submissionId`
Reply	non-zero	non-zero root submission reference

Contract-level validation is intentionally minimal. Curators and clients perform full semantic validation and may reject malformed announcements even if they are present on-chain.

6 Curator Model and Thread Materialization

The reference curator loop is:

1. poll chain events with confirmation depth,
2. fetch immutable submission/content objects from Swarm,
3. validate schema and cross-object consistency,
4. index by announcement order `(blockNumber, logIndex)`,
5. publish fresh `boardIndex`, `threadIndex`, and `globalIndex`,
6. update stable feed pointers,
7. re-publish `curatorProfile` and emit `CuratorDeclared` when board feed set changes.

In v1 reference behavior, ordering is chronological by announcement order (newest first), with no scoring or moderation policy beyond protocol validation.

7 Client Flows

7.1 Read Path: Board and Thread

Read without wallet: the SPA can browse with read-only RPC + Swarm reads, without signing transactions.

7.2 Write Path: Post or Reply Publish

Write with wallet: publication requires Swarm publish capability and a wallet transaction for on-chain announcement.

Implementation note: first-time authors need a stable `userFeed` manifest before their first publish, so authored objects can point to a durable feed identity and subsequent `userFeedIndex` updates can target the same feed.

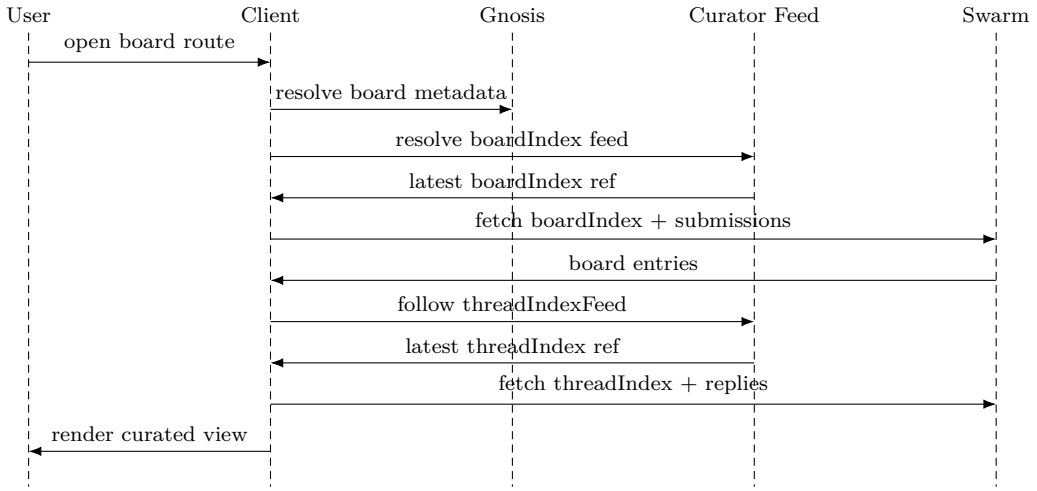

Figure 2: Board/thread read sequence through curator-backed views.

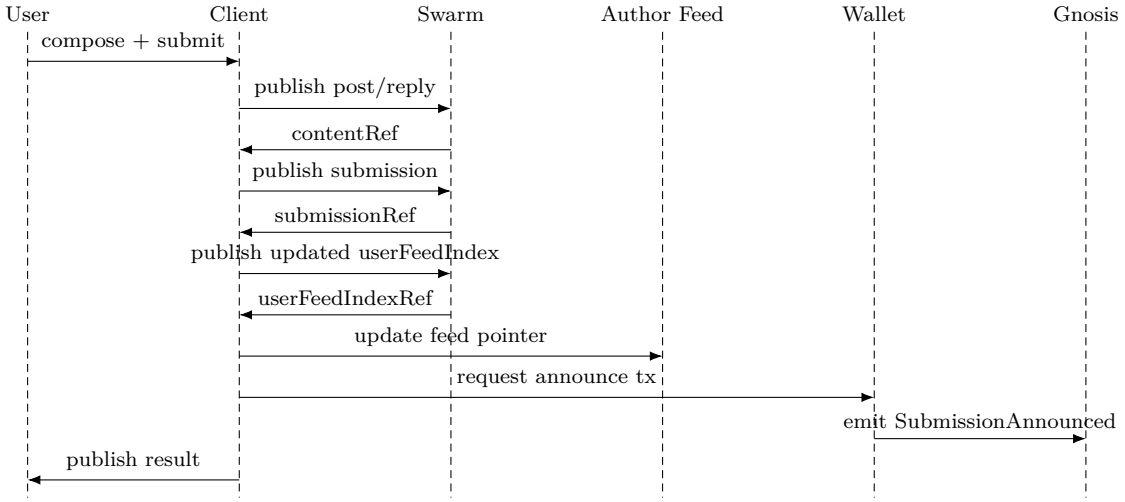

Figure 3: Post/reply publish sequence for v1 clients.

8 Security and Failure Assumptions

Swarmit v1 assumes and explicitly handles the following:

- **Malformed announcements:** on-chain logs may reference invalid objects; curators and clients must validate and filter.
- **Propagation lag:** recent feed updates may temporarily resolve to stale content or 404 until network propagation completes.
- **Curator outages:** missing curator data should trigger client fallthrough to alternate curators when available.
- **Infrastructure dependencies:** Bee availability, postage batches, and RPC reliability affect liveness.

- **Spam/abuse limits:** v1 does not fully solve anti-spam economics or governance-grade moderation.

9 Reference Implementation Status (Current)

As of **March 25, 2026**, the protocol is beyond design-only status and has a functioning reference stack.

9.1 Swarmit SPA

Current SPA status:

- static, Swarm-hostable client with hash routes,
- board, thread, submit, board-create, curator-picker, and profile views,
- debug routes for Swarm read verification and fixture creation,
- curator selection with fallback behavior,
- multi-step publish pipeline with progress and partial-success handling.

Representative routes include:

- `#/`
- `#/r/<slug>`
- `#/r/<slug>/submit`
- `#/r/<slug>/comments/<rootSubmissionId>`
- `#/curators`
- `#/create-board`
- `#/debug/swarm-read` and `#/debug/create-fixtures`

9.2 On-Chain Deployment

- Contract: `SwarmitRegistry`
- Chain: Gnosis (chain ID 100)
- Deployed address: `0x34b27b9978E05B6EfD8AFEcc133C3b1fC5431613`
- Documented deploy block for curator catch-up: `45315302`

9.3 Curator Service

The Swarmit Curator reference implementation is operational as a Node.js background worker:

- polls chain events with confirmation depth,
- validates submission/content objects before indexing,
- builds and publishes `boardIndex`, `threadIndex`, `globalIndex`,
- updates feed manifests and republishes `curatorProfile` when needed,
- persists state as a rebuildable cache.

9.4 End-to-End MVP Loop

Manual smoke testing confirms the full reference loop:

1. publish fixture or real post from SPA,
2. announce on-chain,
3. curator ingests and materializes indexes,
4. board and thread views render the new content in SPA.

This demonstrates practical viability of the v1 publish/read pipeline, not just protocol completeness on paper.

10 Operations and Deployment Reality

10.1 SPA Runtime Requirements

- Freedom Browser for integrated `window.swarm` + wallet flows,
- optional read-only RPC for browsing without wallet,
- local vendored JS dependencies due Swarm-hosted CSP constraints.

10.2 Curator Runtime Requirements

- Bee API endpoint and valid postage batch,
- funded Gnosis wallet for `CuratorDeclared` gas,
- private key used consistently for feed signing and chain declaration identity,
- persistent state file for resumption and operational continuity.

11 Future Work and v2 Directions

Key directions beyond v1 include:

- richer moderation and attestation signaling,
- stronger anti-spam mechanisms,
- curator policy diversity and reputation layers,
- optional feed-backed mutable curator profiles,
- optimized Swarm storage paths (for example, coordinated bytes-path reads where ecosystem support exists),
- hardening for reorg, downtime, and large-scale indexing operations.

12 Conclusion

Swarmit v1 combines immutable authorship, public chain coordination, and competitive curation into a coherent discussion protocol with a working reference implementation. The system's practical strength is not elimination of all infrastructure dependencies; it is the removal of a single mandatory operator from publication and reading.

With a deployed contract, functioning SPA, and running curator implementation, Swarmit has progressed from specification to operational MVP while preserving clear upgrade paths for governance, moderation, and scaling in future versions.

A Event Signature Appendix

```
BoardRegistered(bytes32 indexed boardId, string slug, string boardRef, address
governance)

BoardMetadataUpdated(bytes32 indexed boardId, string boardRef)

SubmissionAnnounced(bytes32 indexed boardId, bytes32 indexed submissionId, string
submissionRef,

  bytes32 parentSubmissionId, bytes32 rootSubmissionId, address author)

CuratorDeclared(address indexed curator, string curatorProfileRef)
```

B Reference Documents

Protocol and schema references:

- Protocol v1 Spec: `docs/swarm-message-board-v1-spec.md`

- v1 Schemas: `docs/swarm-message-board-v1-schemas.md`
- v1 Contract Spec: `docs/swarm-message-board-v1-contract-spec.md`
- Swarmit repository: https://github.com/flotob/swarmit

Implementation and operations references:

- Swarmit README (Swarmit repo path: `README.md`)
- Swarmit Curator README (Curator repo path: `README.md`)
- Swarmit Curator Architecture (Curator repo path: `docs/architecture.md`)
- Swarmit Curator Roadmap (Curator repo path: `docs/roadmap.md`)
- Swarmit Curator repository: https://github.com/flotob/swarmit-curator